$\mathrm{SPHINCS}^+_\mathrm{B}$: 署名処理をバッチ化した SPHINCS$^+$

三好 茜音[1,a)]　高橋 康[1]　縫田 光司[2]　矢内 直人[1]

概要：SPHINCS$^+$ は NIST によって耐量子計算機暗号の標準化方式に選定されたハッシュベース署名であるが，署名サイズが大きい．このサイズを抑えるために署名をバッチ化することも考えられるが，ハッシュベース署名の代数構造を通じて安全なバッチ化ができるかは非自明である．本稿では，ハッシュベース署名の構造を通じて署名をバッチ化する $\mathrm{SPHINCS}^+_\mathrm{B}$ を提案する．本稿の主な貢献は $\mathrm{SPHINCS}^+_\mathrm{B}$ の構成に関して，新たな概念として鍵付きハッシュ関数 MKHF を導入し，内包するマークル木の葉の選択を抽象化したアルゴリズムを設計したことにある．これにより，FORS 署名に関して同じ公開鍵の下で複数の署名を一括して構成できるようになる．$\mathrm{SPHINCS}^+_\mathrm{B}$ の安全性は，本稿による新たな証明技法として，上述した MKHF および従来のハッシュ関数の衝突困難性の下で証明できる．実装実験を通じて $\mathrm{SPHINCS}^+_\mathrm{B}$ が SPHINCS$^+$ を個別に実行するよりも処理時間を 95.7%，署名サイズを 62.8% 以上削減できることを示す．

キーワード：SPHINCS$^+$，バッチ署名，ブロックチェーン, 耐量子計算機暗号

Batch Signatures for SPHINCS$^+$

Akane Miyoshi[1,a)]　Yasushi Takahashi[1]　Koji Nuida[2]　Naoto Yanai[1]

Abstract: SPHINCS$^+$is a hash-based signature scheme standardized by NIST as post-quantum cryptography, but it suffers from large signature sizes. Although one might think that batching its signatures to reduce the signature size, it is non-trivial whether a secure batch scheme is feasible. In this paper, we propose $\mathrm{SPHINCS}^+_\mathrm{B}$, a hash-based signature scheme whose algebraic structure is batched. Our primary contributions are to present a novel hash function and to design an algorithm that generalizes selections of leaves in Merkle trees. This enables multiple FORS signatures to be constructed under the same public key. The security of $\mathrm{SPHINCS}^+_\mathrm{B}$is provable under the collision resistance of the above hash function through our novel proof technique. When we evaluate the presented construction through cryptographic implementation, we also demonstrate that it can improve the computation time of the signature by 95.7% and the signature size by 62.8% compared with iterations of SPHINCS$^+$.

Keywords: SPHINCS$^+$, Batch Signatures, Blockchain, Post-Quantum Cryptography

1. はじめに

近年の量子計算機の研究開発に伴い，量子計算機に対しても安全性を保証する耐量子計算機暗号の研究もまた加速しており，世界中で耐量子計算機暗号への移行が検討されている．SPHINCS$^+$ [1] は NIST により 2024 年に標準化対象方式として発表されたハッシュベース署名方式である．安全性はハッシュの衝突困難性に基づいており，複数のハッシュベース署名を連結することで構成されている．SPHINCS$^+$ は Ethereum などブロックチェーンにおいて耐量子計算機暗号への移行対象として注目されている[*1]．

一方，耐量子計算機暗号でそのデータサイズが大きいことに起因して，メモリなどリソースの消費を増加させる [2]．Ethereum ではプログラムの実行時におけるリソース消費に比例して実行に掛かる費用をユーザが支払う必要があるため [3]，上述した点は重要な問題となる．直観的に

[1] パナソニック ホールディングス株式会社
Panasonic Holdings Corporation
[2] 九州大学 マス・フォア・インダストリ研究所
Institute of Mathematics for Industry, Kyushu University
[a)] miyoshi.akane@jp.panasonic.com

[*1] https://ethresear.ch/t/sphincs-minus-efficient-stateless-post-quantum-signature-verification-on-the-evm

© 2026 Information Processing Society of Japan

This paper is work in progress and not peer-reviewed.

は，耐量子計算機暗号によりリソースを多く消費することで，Ethereum においては仮想通貨を多く消費してしまう．

このリソース消費を抑える方式として，本稿ではバッチ署名 [4, 5] に注目する．バッチ署名は複数の平文を葉に持つ木構造を構成し，その根に対して署名をすることで，全体のデータサイズの削減を図る方式である．耐量子計算機暗号の負荷を削減する目的で，バッチ署名は Google[*2]や Let's Encrypt[*3]などにおいてサービスへの導入が進んでいる．

しかしながら，既存のバッチ署名は任意の署名方式に対する一般的構成であり，ハッシュベース署名に特化した構成ではない．バッチ署名もハッシュベース署名も木構造に基づいて構成されるため，その構成を考慮することで署名サイズの削減や計算処理の削減が期待できる [6]. 本稿では以下の問いを明らかにする: **SPHINCS$^+$ の構成を活用することで，効率的かつ安全なバッチ署名が得られるか**.

1.1 本稿の貢献

本稿では，上述した問いの答えとして SPHINCS^+_B を提案する．SPHINCS^+_B は基となる SPHINCS$^+$ の構成のうち連結における最下部の方式となる FORS 署名を，既存のバッチ署名におけるマークル木と見做す．これにより FORS 署名の上位層にある全ての署名を単一の署名として共通化することで，全体のデータサイズを大幅に削減できる．

SPHINCS^+_B の構成は, 基となる SPHINCS$^+$ の構成に起因して非自明である．SPHINCS$^+$ では署名の代数構造としてハッシュ値からなる複数のマークル木を構成し，平文から得られるハッシュ値を用いて，どのマークル木のどの葉を使用するか決定する．一方，SPHINCS^+_B では，平文集合から得られるハッシュ値の部分文字列を抽出することで使用するマークル木を決定したのち，各平文から得られるハッシュ値の部分文字列を抽出することで, そのマークル木において使用する葉を決定する．このとき，各平文に対する一般的なハッシュ値の衝突困難性はもちろん，それらの部分文字列にも衝突困難性が求められる．このような性質は，一般的なハッシュ関数が持たない性質となる．

本稿の技術的貢献は，上述した非自明さの解決として，入力の平文集合に対して，集合から得られるハッシュ値の部分文字列と各平文から得られるハッシュ値の部分文字列が同時に衝突しないことを包含する新たな概念として平文集合に対する鍵付きハッシュ関数 MKHF を定義したこと，また，MKHF を通じてマークル木における葉の選択を抽象化することで，安全性をその衝突困難性に帰着できる構成を示したことにある．MKHF の効率的な構成については今後の課題としているが，上述した MKHF の定義の下で，ゲーム列を通じた新たな証明技法も示している．

[*2] https://blog.google/security/cultivating-a-robust-and-efficient-quantum-safe-https/
[*3] https://letsencrypt.org/2026/06/03/pq-certs

1.2 関連研究

SPHINCS$^+$ の応用として著者の知る限り SPHINCS-α [7], SPHINCS-C [8], SPHINCS-minus [9], HRSS [10] がある．SPHINCS-α [7] と SPHINCS-C [8] では SPHINCS$^+$ の要素技術となる WOTS 署名と FORS 署名の構成を変えることで，効率化を図った. また，SPHINCS-minus [9] は Ethereum での利用を想定して署名回数を制限したパラメータを採用し，Ethereum 上の検証コストを削減した．これらの研究における課題と貢献は本稿と異なるが, SPHINCS^+_B と融合することで更なる効率化が期待できる．一方，HRSS [10] は匿名性を持つ署名を SPHINCS$^+$ を応用して設計している．これは SPHINCS^+_B とは異なる機能を扱っている．

バッチ署名 [5] は木構造を通じて任意の署名方式を効率的に扱う一般的方式として IETF にて最初に提案され, その安全性は Aguilar-Melchor ら [4] が証明した．これらは二分木に基づいている．マークル木を通じて任意の署名に対し負荷を削減する手法 [11] もあるが，ステートフルな署名となる．著者の先行研究 [12] がステートレスかつマークル木を実現するが，上述した一連の方式では本稿のようなハッシュベース署名に特化した検討はされていない．

最後にブロックチェーンと耐量子計算機暗号の境界領域として, poqeth [13] は Ethereum に向けて設計した耐量子計算機暗号ライブラリである．Bella ら [14] は標準化されている署名方式の中では SPHINCS$^+$ が最も性能が低いことを示した．これらの研究は Ethereum における耐量子計算機暗号の実装であり，本稿のような高機能暗号の機能は持たない．本稿に近い既存研究として，Drake ら [15] はブロックチェーンには SPHINCS$^+$ よりも XMSS が適しているとして，XMSS ベース方式の多重署名を構成した．多重署名は署名の負荷を削減する意味でバッチ署名に近い技術であるが, 本稿はこの方式とは基となる方式が異なり，また，本稿で議論するような課題は検討されていない．

2. 準備

本節では，擬似ランダム関数，鍵付きハッシュ関数，tweakable ハッシュ関数を定義する．

2.1 擬似ランダム関数

擬似ランダム関数 (PRF) とは，鍵空間 $\mathcal{K}$， 入力空間 $\mathcal{X}$，出力空間 $\mathcal{Y}$ 上の PRF : $\mathcal{K}\times\mathcal{X}\to\mathcal{Y}$ である．

定義 1 (PRF) 関数族 F が PRF を持つとは，以下に定義するゲームにおいて，攻撃者 $\mathcal{A}$ の勝利する確率 $\mathsf{Adv}^{\mathrm{PRF}}_{\mathrm{F},\mathcal{A}} := |\Pr[\mathcal{A}\text{ wins PRF}] - 1/2|$ が無視可能であることをいう．

Setup: 挑戦者 $\mathcal{C}$ は F の鍵 k をランダムに生成し，ビット $b\in\{0,1\}$ をランダムに選択する; **Oracle**: $\mathcal{A}$ は任意の入力値 x を選び $\mathcal{C}$ に要求する． $\mathcal{C}$ は $b=0$ の場合 $\mathrm{F}(k,x)$ を，$b=1$ の場合ランダム値を返す; **Guess**:$\mathcal{A}$ は出力値から b を推測し，推測値 b^* を返す．$b=b^*$ なら $\mathcal{A}$ が勝利する．

© 2026 Information Processing Society of Japan

2.2 ハッシュ関数

鍵付きハッシュ関数 (KHF) とは，鍵空間 $\mathcal{K}$，平文空間 $\mathcal{M}$，ハッシュ値空間 $\mathcal{Y}$ 上の KHF : $\mathcal{K} \times \mathcal{M} \to \mathcal{Y}$ である．鍵付きハッシュ関数の安全性は付録 A.1.1 に示す．

また，tweakable ハッシュ関数とは，公開パラメータ空間 $\mathcal{P}$，tweak 空間 $\mathcal{T}$，平文空間 $\mathcal{M}$，ハッシュ値空間 $\mathcal{Y}$ 上の THF : $\mathcal{P} \times \mathcal{T} \times \mathcal{M} \to \mathcal{Y}$ である．tweakable ハッシュ関数の安全性は付録 A.1.2 に示す．

2.3 SPHINCS$^+$

SPHINCS$^+$ は FORS，WOTS$^+$，XMSS，HT から構成されるハッシュベース署名である．以降では全体構成を述べたのち，本稿の主な議論の対象となる FORS と HT を述べる．

2.3.1 全体構成

SPHINCS$^+$ は，後述する FORS, XMSS, HT を階層化することで構成される．また，XMSS と HT において核となる署名はワンタイム署名である WOTS+ として生成される．まず平文 μ をハッシュ関数によりダイジェストに変換し，FORS により当該ダイジェストに対する署名を生成する．次に，FORS の公開鍵に対して HT に含まれる XMSS を用いて段階的に署名を行った後，最上位の公開鍵に対応付ける．検証では，FORS から公開鍵を再構成した後，それを入力として HT の各層の XMSS 署名を順に検証することで，公開鍵と一致するかを確認する．

SPHINCS$^+$ ではセキュリティパラメータを λ, HT の高さを h, 木の層数を d, FORS における Merkle tree の本数を k, FORS における Merkle tree の葉の個数を t とする．また，m は上記のパラメータを用いて以下のように定義される．

$$m = \left\lfloor \frac{k \log t + 7}{8} \right\rfloor + \left\lfloor \frac{h - h/d + 7}{8} \right\rfloor + \left\lfloor \frac{h/d + 7}{8} \right\rfloor$$

2.3.2 FORS

Forest of Random Subsets（FORS）は ハッシュ関数の出力長 $k \log t$ ビットのダイジェストに対し署名する．秘密鍵は n バイトの値を kt 個生成し，これを k 個の集合に分割し構成する．これらの値は SPHINCS$^+$ の秘密鍵に含まれるシードから擬似ランダムに生成される．各秘密鍵要素に tweakable ハッシュ関数を適用し，マークル木の葉とする．公開鍵は，各集合に対する k 本のマークル木の根をハッシュ化して得られる n バイトの値である．本稿では，FORS 公開鍵を根とし，その下に k 本のマークル木を持つ構造全体を FORS 木と呼ぶ．署名生成では，ダイジェストを k 個の $\log t$ ビット列に分割し，各値に対応する秘密鍵要素とマークル木の根を再構築するための認証パスを出力する．検証では，各マークル木の根を再構成し，公開鍵を計算する．得られた公開鍵が一致すれば検証は成功とする．

2.3.3 HT

Hypertree（HT）は XMSS を多層に拡張した構造であり，複数の XMSS をマークル木上に接続することで構成される．HT は d 層の XMSS から構成され，各 XMSS は高さ $h' = h/d$ のマークル木を持つ．最下層の XMSS は実際の署名対象に対して署名を行う．SPHINCS$^+$ においては平文に対応する FORS 木の根（FORS の公開鍵）に対して署名を行う．この FORS 木を FORS のインスタンスと呼ぶ．一方，上位の XMSS は，下位層の XMSS の根に対して署名を行う．このようにして，各層の XMSS は下位層の公開鍵を認証し，最終的に最上位の根が全体の公開鍵となる．検証では，まず最下層の XMSS 署名により公開鍵を再構成し，その結果を次の層の XMSS 検証の入力として用いる．これを上位層へ順に繰り返すことで，最終的に最上位の公開鍵と一致するかを確認し，一致する場合に検証は成功する．

2.4 バッチ署名

バッチ署名の構文と安全性を示す．本稿の定義は先行研究 [4, 12] に従い，複数の平文に対する署名を一括して生成・検証できる一方，単一の平文からなるバッチ署名は個別のデジタル署名と同等の意味を持つ. バッチ署名は KeyGen, BSign，BVerify の 3 つのアルゴリズムで構成される．

KeyGen はセキュリティパラメータ λ，平文集合の最大要素数 $N_{\max}$ を入力として，秘密鍵 SK と公開鍵 PK のペアを出力する; **BSign** は秘密鍵 SK と平文集合 $M = \{\mu_i\}_{i=0}^{N-1}$ を入力として，署名 S を出力する; **BVerify** は公開鍵 PK，平文集合 $M = \{\mu_i\}_{i=0}^{N-1}$ と署名 S を入力として，ビット $\{0, 1\}$ を出力する．

バッチ署名が正当性を持つとは，任意の $(\mathrm{PK}, \mathrm{SK}) \leftarrow \mathrm{KeyGen}(\lambda, N_{\max})$ と，要素数が $N \le N_{\max}$ の任意の平文集合 $M = \{\mu_i\}_{i=0}^{N-1}$ に対して，$\mathrm{BVerify}(\mathrm{PK}, M, \mathrm{BSign}(\mathrm{SK}, M)) = 1$ が成立することを言う．

バッチ署名の自明な構成はディジタル署名の処理 Sign を平文集合の要素数回実行することだが，その構成が持たない性質として，効率性が求められる．バッチ署名が効率性を持つとは，任意のセキュリティパラメータ λ と要素数が $N \le N_{\max}$ の任意の平文集合 $M = \{\mu_i\}_{i=0}^{N-1}$ に対して，$S \leftarrow \mathrm{BSign}(\mathrm{SK}, M)$ および $S'_i \leftarrow \mathrm{Sign}(\mathrm{SK}, \mu_i)$ とするとき，$|S| < \sum_{i=0}^{N-1} |S'_i|$ となることを言う．

バッチ署名の安全性は，バッチ署名の偽造不可能性（Batch Existential Unforgeability under Chosen Message Attack, BEUF-CMA）で定義される．バッチ署名 TBS が BEUF-CMA を持つとは．以下に定義するゲームにおいて, 任意の確率的多項式時間攻撃者 $\mathcal{A}$ の勝利する確率 $\mathsf{Adv}^{\mathrm{BEUF-CMA}}_{\mathrm{TBS},N,\mathcal{A}}(\lambda)$ が λ に対して無視可能であることを言う．

Setup: セキュリティパラメータ λ を与えられた挑戦者 C は $\mathrm{KeyGen}(\lambda, N_{\max})$ を実行し，出力された鍵ペアのうち公開鍵 PK を $\mathcal{A}$ に渡す．また，$\mathcal{M} = \emptyset$ とする; **Oracle**:$\mathcal{A}$ は要素数が $N_i \le N_{\max}$ の任意の平文集合 $M_i = \{\mu_{(i,0)}, \mu_{(i,1)}, \cdots, \mu_{(i,N_i-1)}\}$ の署名を要求する．挑戦者は $\mathrm{BSign}(\mathrm{SK}, M_i)$ を実行し, 出力である署名 S_i を返す．こ

© 2026 Information Processing Society of Japan

のとき, $\mathcal{M} = \mathcal{M} \bigcup \{M_i\}$ とする; **CollisionTest**:$\mathcal{A}$ は要素数が $N^* \leq N_{\max}$ の任意の平文集合 $M^* = \{\mu_0^*, \mu_1^*, \cdots, \mu_{N^*-1}^*\}$ とそれに対応する署名 S^* を出力する．このとき, 以下が全て成り立つ場合, $\mathcal{A}$ は勝利する:$M^* \notin \mathcal{M}$, $\mathrm{BVerify}(\mathrm{PK}, M^*, S^*) = 1$.

3. 平文集合に対する鍵付きハッシュ関数

平文集合に対する鍵付きハッシュ関数 (MKHF) を定義する．1.1 節で述べた通り，本節は本稿の貢献の一部である．任意の整数 N, L_1, L_2 において, 鍵空間 $\mathcal{K}$, 平文空間 $\mathcal{M}^N$, 出力空間 $\mathcal{D} = \mathcal{D}_1^{L_1} \times \mathcal{D}_2^{L_2}$ 上の $\mathrm{MKHF} : \mathcal{K} \times \mathcal{M}^N \to \mathcal{D}$ である．直観的には以下に述べる具体的手法が考えられる．まず鍵 k, 平文集合 $M = \{\mu_0, \cdots, \mu_{N-1}\}$ を入力とし, 各平文および平文集合全体に対し, $y_i = \mathrm{KHF}(k, \mu_i)$, $y = \mathrm{KHF}(k, M)$ を計算する．次に, 各ハッシュ値を上位ビット列と下位 a ビットに分割し, $md_i = \lfloor y_i / 2^a \rfloor$, $md_N = \lfloor y / 2^a \rfloor$, $\mathrm{index} = y \pmod{2^a}$ を求める．最後に，$MD = \{md_0, \ldots, md_{N-1}, md_N\}$ とし，$D = (MD, \mathrm{index})$ を平文集合に対する鍵付きハッシュ関数の出力とする．上述した構成が真に MKHF の要件を満たすか含めてその効率的な構成は今後の課題であるものの, MKHF の安全性は以下の通り定義される．

定義 2 (MITSR) 平文集合に対する鍵付きハッシュ関数 (MKHF) が MITSR を持つとは，以下に定義するゲームにおいて，攻撃者 $\mathcal{A}$ の勝利する確率 $\mathsf{Adv}^{\mathrm{MITSR}}_{\mathrm{MKHF},\mathcal{A}}$ が無視可能であることをいう．

Setup 挑戦者 C は $Q = \emptyset$ を初期化する．

Oracle $\mathcal{A}$ は任意の平文集合 $M = \{\mu_0, \cdots, \mu_{N-1}\} \in \mathcal{M}^N$ を選び C に要求する．C は鍵 k をランダムに生成し, (k, M) を Q に追加した上で，鍵 k を $\mathcal{A}$ に返す.

CollisionTest $\mathcal{A}$ は (k^*, M^*) を出力する．このとき，$(k^*, M^*) \notin Q$ かつ $\mathrm{MAP}(\mathrm{MKHF}(k^*, M^*)) \cap \{\mathrm{MAP}(\mathrm{MKHF}(k_i, M_i)) \mid (k_i, M_i) \in Q\} \neq \emptyset$ が成り立つとき $\mathcal{A}$ が勝利する．MAP は鍵付きハッシュ関数の出力をターゲット集合へ写像する関数で，MITSR の定義において攻撃対象となる要素集合を規定する．

4. $\mathrm{SPHINCS}^+_{\mathrm{B}}$ の提案

本節では, 提案方式となる $\mathrm{SPHINCS}^+_{\mathrm{B}}$ の詳細を述べる．平文集合の要素数を N, バイトの集合を $\mathbb{B}$ とする．$\mathrm{SPHINCS}^+_{\mathrm{B}}$ では, tweakable ハッシュ関数 $\mathrm{T}_l : \mathbb{B}^n \times \mathbb{B}^{32} \times \mathbb{B}^{ln} \to \mathbb{B}^n$, 疑似ランダム関数 $\mathrm{MKG} : \mathbb{B}^n \times \mathbb{B}^n \times \mathbb{B}^* \to \mathbb{B}^n$，平文集合に対する鍵付きハッシュ関数 $\mathrm{MCCO} : \mathbb{B}^n \times \mathbb{B}^n \times \mathbb{B}^n \times \mathbb{B}^{*\times N} \to \mathbb{B}^{a \times N} \times \mathbb{B}^b$ を用いる．ここで，$a = \lfloor (k \log t + 7)/8 \rfloor$, $b = \lfloor (h - h/d + 7)/8 \rfloor + \lfloor (h/d + 7)/8 \rfloor$ とする.

4.1 全体像

$\mathrm{SPHINCS}^+_{\mathrm{B}}$ では複数の平文から構成される集合に対して同一の FORS 構造を共有して署名を生成する．具体的には，平文集合に対する鍵付きハッシュ関数 (MKHF) を用いて，平文集合から共有して用いる FORS のインスタンスを決定し，各々に対応する FORS 署名を生成する．その後, 生成された FORS 公開鍵に対して HT 署名を付与する．検証時には，平文集合および各平文に対応する FORS 署名から FORS 公開鍵を再構成し，それら全てから同一の FORS 公開鍵が得られることを確認し，当該公開鍵に対する HT 署名を検証する．これにより，平文集合全体の署名を効率的に検証することができる.

4.2 アルゴリズム

$\mathrm{SPHINCS}^+_{\mathrm{B}}$ について KeyGen を Algorithm 1，BSign を Algorithm 2，BVerify を Algorithm 3 にそれぞれ示す．以下では，各アルゴリズムの処理内容について説明する.

Algorithm 1 の KeyGen は，セキュリティパラメータ λ と平文集合の最大要素数 $N_{\max}$ を入力とする．$\mathrm{SPHINCS}^+$ の鍵生成アルゴリズムにより公開鍵 PK と秘密鍵 SK を生成する．このとき，SK を生成する際に使用する疑似ランダム関数を SKG とする．最後に，鍵ペア (PK,SK) を出力する.

Algorithm 2 の BSign は，秘密鍵 SK と平文集合 $M = \{\mu_0, \cdots, \mu_{N-1}\}$ を入力とする．まず，乱数 S_R を生成し，これを公開鍵に含まれるシード値および平文集合 M とともに MCCO へ入力する．これにより，FORS に対する平文集合 MD と，利用する FORS インスタンスを示すインデックス index を算出する．ここで，MD は，各平文に対応する値 $MD[i]$ $(i \in [0, N-1])$ と，平文集合に対する値 $MD[N]$ から構成される．index は，idx_tree, idx_leaf に分解される．次に，MD を入力として FORS 署名 $S_{\mathrm{FORS}} = \{\mathrm{sig}_0, \cdots, \mathrm{sig}_{N-1}, \mathrm{sig}_N\}$ と FORS の公開鍵 $\mathrm{PK}_{\mathrm{FORS}}$ を生成する．最後に，$\mathrm{PK}_{\mathrm{FORS}}$ に対して，HT 署名 S_{HT} を生成し，署名として $S = \{S_R, S_{\mathrm{FORS}}, S_{\mathrm{HT}}\}$ を出力する.

Algorithm 3 の BVerify は，公開鍵 PK, 平文集合 $M = \{\mu_0, \cdots, \mu_{N-1}\}$，署名 $S = \{S_R, S_{\mathrm{FORS}}, S_{\mathrm{HT}}\}$ を入力とする, ここで $S_{\mathrm{FORS}} = \{\mathrm{sig}_0, \cdots, \mathrm{sig}_{N-1}, \mathrm{sig}_N\}$ である．まず，署名 S_R，公開鍵に含まれるシード値および平文集合 M を MCCO へ入力することで，FORS に対する平文集合 MD と，利用する FORS インスタンスを示すインデックス index を算出する．インデックス index は，idx_tree，idx_leaf に分解される．次に，MD の各要素と対応する FORS 署名 $\mathrm{sig}_i \in S_{\mathrm{FORS}}$ から FORS の公開鍵 $\mathrm{PK}_{\mathrm{FORS}}$ を再構成する．再構成された公開鍵が一致しない場合，検証は失敗となる．一致した場合は，再構成された $\mathrm{PK}_{\mathrm{FORS}}$ を入力として，HT の検証アルゴリズムを実行し，得られた結果を b とし，これを検証結果として出力する.

補足. 上述した Bverify は署名に関して，BSign から出力された署名集合のうち一部の署名だけを検証する場合も対応可能である．具体的には，Algorithm 3 の 5–9 行目における処理を，検証の対象となる署名 S に含まれる S_{FORS} に

© 2026 Information Processing Society of Japan

Algorithm 1: KeyGen

Input: λ, $N_{\max}$
Output: PK, SK
PK, SK $\leftarrow$ SPHINCS$^+$.KeyGen(λ).

Algorithm 2: BSign

Input: SK, $M = \{\mu_0, \cdots, \mu_{N-1}\}$
Output: $S = \{S_R, S_{\mathrm{FORS}}, S_{\mathrm{HT}}\}$
opt $\leftarrow \mathbb{B}^n$.
$S_R \leftarrow$ MKG (SK.prf, opt, M).
MD, index $\leftarrow$ MCCO (S_R, PK.seed, PK.root, M).
idx_tree,idx_leaf $\leftarrow \lfloor \mathrm{index}/2^{(h/d)} \rfloor$, index $\pmod{2^{(h/d)}}$.
Initialize ADRS for FORS with idx_tree and idx_leaf.
S_{FORS}, $\mathrm{PK}_{\mathrm{FORS}} \leftarrow$ FORS.BSign(MD, SK.seed, PK.seed, ADRS).
Initialize ADRS for HT.
$S_{\mathrm{HT}} \leftarrow$ HT.Sign($\mathrm{PK}_{\mathrm{FORS}}$, SK.seed, PK.seed, idx_tree, idx_leaf).

Subroutine: FORS.BSign
Input: $MD = \{md_0, \cdots, md_{N-1}, md_N\}$, SK.seed, PK.seed, ADRS
Output: $S_{\mathrm{FORS}} = \{\mathrm{sig}_0, \cdots, \mathrm{sig}_{N-1}, \mathrm{sig}_N\}$, $\mathrm{PK}_{\mathrm{FORS}}$
for $0 \le i < k$ **do**
  for $0 \le j \le N$ **do**
    idx$[i][j] \leftarrow i$-th block of $\log t$ bits of md_j.
root $\leftarrow \epsilon$.
for $0 \le i < k$ **do**
  for $0 \le j \le N$ **do**
    $\mathrm{sig}_j \leftarrow \mathrm{sig}_j \parallel$ FORS.SKgen(SK.seed, ADRS, it + idx$[i][j]$).
  for $0 \le \ell < t$ **do**
    sk $\leftarrow$ FORS.SKgen(SK.seed, ADRS, $it + \ell$).
    node $\leftarrow \mathrm{T}_1$(PK.seed, ADRS, sk).
    for $0 \le h < \log t$ **do**
      $\ell' \leftarrow \lfloor \ell/2^h \rfloor$.
      for $0 \le j \le N$ **do**
        if $\ell' \oplus \lfloor \mathrm{idx}[i][j]/2^h \rfloor = 1$ **then**
          $\mathrm{sig}_j \leftarrow \mathrm{sig}_j \parallel$ node.
      if $\ell' \bmod 2 = 0$ **then**
        Stack.push(node).
        break
      Initialize ADRS for FORS with $(h+1)$ and $\lfloor \ell/2^{h+1} \rfloor$.
      left $\leftarrow$ Stack.pop().
      node $\leftarrow \mathrm{T}_2$(PK.seed, ADRS, (left $\parallel$ node)).
  root $\leftarrow$ root $\parallel$ node.
$\mathrm{PK}_{\mathrm{FORS}} \leftarrow \mathrm{T}_k$(PK.seed, ADRS, root)

ついて，検証対象となる任意の部分集合 $S'_{\mathrm{FORS}} \subset S_{\mathrm{FORS}}$ に対応する $\mathrm{sig}_i \in S'_{\mathrm{FORS}}$ にだけ実行すればよい．

5. SPHINCS$^+_{\mathrm{B}}$ における安全性証明

本節では，SPHINCS$^+_{\mathrm{B}}$ の安全性を証明する．本稿の証明は MKHF に安全性を帰着させる，独自の証明技法となる．

定理 1 (BEUF-CMA) 秘密鍵生成に使用される疑似ランダム関数 SKG および MCCO の計算に用いる鍵の生成に使用される疑似ランダム関数 MKG が PRF を持つとする．MCCO が MITSR を持つとする．さらに，tweakable ハッシュ関数 T_1 は SM-DT-TCR と SM-DT-DSPR を，T_2 と T_k は SM-DT-TCR-C を持つとする．このとき，SPHINCS$^+_{\mathrm{B}}$ は BEUF-CMA を満たす．

Algorithm 3: BVerify

Input: PK, $M = \{\mu_0, \cdots, \mu_{N-1}\}$, $S = \{S_R, S_{\mathrm{FORS}}, S_{\mathrm{HT}}\}$
Output: b
$MD \leftarrow []$.
MD, index $\leftarrow$ MCCO (S_R, PK.seed, PK.root, M).
idx_tree,idx_leaf $\leftarrow \lfloor \mathrm{index}/2^{(h/d)} \rfloor$, index $\pmod{2^{(h/d)}}$.
Initialize ADRS for FORS with idx_tree and idx_leaf.
$\mathrm{PK}_{\mathrm{FORS}} \leftarrow$ FORS.pkFromSig(sig_N, $MD[N]$, PK.seed, ADRS).
for $0 \le i < N$ **do**
  $\mathrm{PK'}_{\mathrm{FORS}} \leftarrow$ FORS.pkFromSig(sig_i, $MD[i]$, PK.seed, ADRS).
  if $\mathrm{PK}_{\mathrm{FORS}} \neq \mathrm{PK'}_{\mathrm{FORS}}$ **then**
    $b \leftarrow 0$. **exit**
Initialize ADRS for HT.
$b \leftarrow$ HT.Verify($\mathrm{PK}_{\mathrm{FORS}}$, S_{HT}, PK.seed, idx_tree, idx_leaf, PK.root).

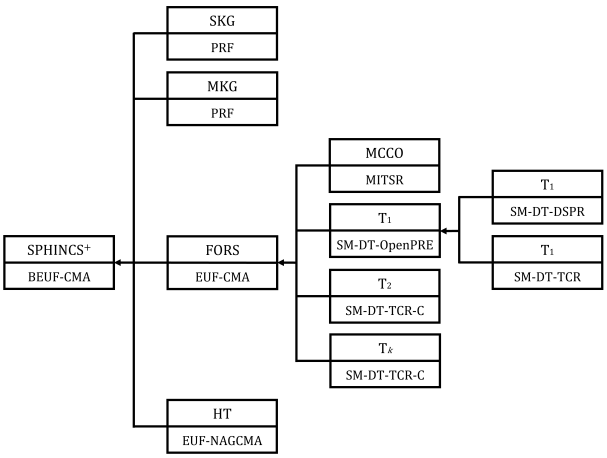


図 1: BEUF-CMA 概要

Proof. SPHINCS$^+_{\mathrm{B}}$ に対する BEUF-CMA の概要を図 1 に示す．図 1 には，HT 部分に対する帰着の流れを含めて示すことができるが，これらは既存研究 [16] と同様であるため，本稿では図示を省略する．

攻撃者 $\mathcal{A}$ は SPHINCS$^+_{\mathrm{B}}$ に対する BEUF-CMA のゲームに勝利すると仮定する．まず，SPHINCS$^+_{\mathrm{B}}$ において用いられる擬似乱数生成を真の乱数に置き換えた方式 SPHINCS$^{+\$}_{\mathrm{B}}$ を考える．この変換は，秘密鍵生成に使用される鍵付きハッシュ関数 SKG および平文集合圧縮と平文圧縮に用いる鍵の生成に使用される鍵付きハッシュ関数 MKG を対応する真のランダム関数で置換することに対応する．攻撃者 $\mathcal{A}$ がこの置換を識別できる場合，SKG または MKG の PRF を破る攻撃者を構成できる．したがって，

$$\left| \mathsf{Adv}^{\text{BEUF-CMA}}_{\text{SPHINCS}^{+\$}_{\mathrm{B}},\mathcal{A}} - \mathsf{Adv}^{\text{BEUF-CMA}}_{\text{SPHINCS}^{+}_{\mathrm{B}},\mathcal{A}} \right| \le \mathsf{Adv}^{\text{PRF}}_{\text{SKG},\mathcal{A}_1} + \mathsf{Adv}^{\text{PRF}}_{\text{MKG},\mathcal{A}_2} \tag{1}$$

が成り立つ．以降，真の乱数を用いる方式 SPHINCS$^{+\$}_{\mathrm{B}}$ に対する BEUF-CMA を考える．

© 2026 Information Processing Society of Japan

$\text{SPHINCS}_{\text{B}}^{+\$}$ における BEUF-CMA における攻撃者 $\mathcal{A}$ の勝利事象は，以下のように分類される．

$\text{Case}_{\text{FORS}}$： $\mathcal{A}$ が出力した FORS 署名 S_{FORS} が，過去のバッチ署名オラクルの応答に含まれていない場合．このとき，FORS に対する偽造が成功することを意味する．$\mathcal{A}$ を用いて，FORS の EUF-CMA のゲームに勝利する攻撃者 $\mathcal{A}_3$ を構成するとすると，$\mathcal{A}_3$ の勝利事象は，以下のように分類される．

- **$\text{Case}_{\text{FORS-1}}$：** $\mathcal{A}_3$ が出力した FORS 署名に含まれる秘密鍵要素 (Algorithm 2 の 3-4 行目) が，過去のバッチ署名オラクル応答により公開された秘密鍵要素のみから構成される場合．
- **$\text{Case}_{\text{FORS-2}}$：** $\text{Case}_{\text{FORS-1}}$ に該当せず，$\mathcal{A}_3$ が出力した FORS 署名から導出される葉ノード (Algorithm 2 の 18 行目) が，過去のバッチ署名オラクル応答により公開された葉ノードと一致する場合．
- **$\text{Case}_{\text{FORS-3}}$：** $\text{Case}_{\text{FORS-1}}$ および $\text{Case}_{\text{FORS-2}}$ に該当せず，$\mathcal{A}_3$ が出力した FORS 署名から計算されるマークル木の根 (Algorithm 2 の 29 行目) が過去のバッチ署名オラクル応答により公開された根と一致する場合．
- **$\text{Case}_{\text{FORS-4}}$：** $\text{Case}_{\text{FORS-1}}$，$\text{Case}_{\text{FORS-2}}$，$\text{Case}_{\text{FORS-3}}$ のいずれにも該当せず，$\mathcal{A}_3$ が出力した FORS 署名から計算される公開鍵 (Algorithm 2 の 31 行目) が過去のバッチ署名オラクル応答により公開された公開鍵と一致する場合．

Case_{HT}： $\mathcal{A}$ が出力した FORS 署名 S_{FORS} が，過去のバッチ署名オラクルの応答に含まれている場合．このとき，HT に対する偽造が成功することを意味する．

全ケースを踏まえ，$\mathcal{A}$ が勝利する確率は以下である．

$$\begin{aligned}\mathsf{Adv}^{\text{BEUF-CMA}}_{\text{SPHINCS}_{\text{B}}^{+\$},\mathcal{A}} &= \mathsf{Adv}_{\text{Case}_{\text{FORS}}} + \mathsf{Adv}_{\text{Case}_{\text{HT}}} \\ &= \mathsf{Adv}_{\text{Case}_{\text{FORS-1}}} + \mathsf{Adv}_{\text{Case}_{\text{FORS-2}}} + \mathsf{Adv}_{\text{Case}_{\text{FORS-3}}} \\ &\quad + \mathsf{Adv}_{\text{Case}_{\text{FORS-4}}} + \mathsf{Adv}_{\text{Case}_{\text{HT}}} \qquad (2)\end{aligned}$$

$\text{Case}_{\text{FORS-1}}$ の場合，FORS の構成過程において，FORS の秘密鍵と平文を通じた葉の選択 (Algorithm 2 の 3-4，9-11 行目) に対して，公開されている署名から得られる秘密鍵要素のみを用いて 偽造署名が生成されることを意味する．このとき，攻撃者が利用した秘密鍵要素の集合は，過去のバッチ署名オラクルへの問い合わせで公開された 秘密鍵要素の集合に含まれる．ここで，MCCO に対する MAP として CM を定義する．CM は MCCO の出力 MD，index を入力とし，FORS で利用される秘密鍵要素のインデックス集合 A へ写像する関数であり，ここで，$b[i_1 : i_2]$ はビット列 b の i_1 ビット目から $i_2 - 1$ ビット目までを表し，$\text{toint}(b)$ はビット列 b を整数として解釈した値を表す．

$$\begin{aligned}A = \Big\{&(\text{toint}(\text{index}[0 : h/d]), \text{toint}(\text{index}[h/d : h]), i, \\ &\text{toint}(md_j[i \log_2 t : (i+1)\log_2 t])) \;\Big|\; 0 \le i < k,\ 0 \le j < N\Big\}.\end{aligned}$$

したがって，$\text{Case}_{\text{FORS-1}}$ が成立するなら，CM(MD,index) で得られる FORS で利用される秘密鍵要素のインデックスの集合が，過去の問い合わせ CM(MD_i,index_i) の和集合との共通要素を少なくとも 1 つ持つこととなる．しかし，偽造対象平文 M^* は署名オラクルへの過去の問い合わせとは異なるため，これは MCCO の MITSR 性を破る攻撃に対応する．よって，$\text{Case}_{\text{FORS-1}}$ で成功する攻撃者から，MCCO の MITSR 性を破る攻撃者 $\mathcal{A}_{3\text{-}1}$ を構成できる．ここで仮定から MCCO は MITSR を満たすため， そのような事象が発生する確率は無視可能である．したがって，$\text{Case}_{\text{FORS-1}}$ において $\mathcal{A}_3$ が勝利する確率は無視可能である．

$$\mathsf{Adv}_{\text{Case}_{\text{FORS-1}}} \le \mathsf{Adv}^{\text{MITSR}}_{\text{MCCO},\mathcal{A}_{3\text{-}1}}$$

$\text{Case}_{\text{FORS-2}}$ の場合，FORS の構成過程において，マークル木の葉生成に用いられる T_1 (Algorithm 2 の 18 行目) に対して，未公開の秘密鍵要素から構成された葉が正当な葉と一致することを意味する．既存研究 [16] では，葉生成に用いられる tweakable ハッシュ関数 F（$\text{SPHINCS}_{\text{B}}^{+}$ の T_1 に対応）が，SM-DT-TCR および SM-DT-DSPR を満たすならば SM-DT-OpenPRE も満たすことが示されている．T_1 は SM-DT-TCR および SM-DT-DSPR を満たすと仮定しているため，SM-DT-OpenPRE の安全性が破られる確率も無視可能である．したがって，$\text{Case}_{\text{FORS-2}}$ において $\mathcal{A}_3$ が勝利する確率は無視可能である．

$$\mathsf{Adv}_{\text{Case}_{\text{FORS-2}}} \le \mathsf{Adv}^{\text{SM-DTOpenPRE}}_{\text{T}_1,\mathcal{A}_{3\text{-}2}}$$

$\text{Case}_{\text{FORS-3}}$ の場合，FORS の構成過程において，マークル木の葉から根を構築する過程のいずれかで，ノード生成に用いられる tweakable ハッシュ関数 T_2(Algorithm 2 の 29 行目) に衝突が生じることを意味する．$\text{Case}_{\text{FORS-3}}$ が発生した場合， 同一の公開パラメータおよび tweak に対する T_2 の衝突を抽出できるため， $\text{Case}_{\text{FORS-3}}$ を発生させる攻撃者から SM-DT-TCR-C を破る攻撃者 $\mathcal{A}_{3\text{-}3}$ を構成できる．一方，T_2 は SM-DT-TCR-C を満たすと仮定しているため，$\mathcal{A}_{3\text{-}3}$ の成功確率は無視可能である．したがって，$\text{Case}_{\text{FORS-3}}$ において $\mathcal{A}_3$ が勝利する確率は無視可能である．

$$\mathsf{Adv}_{\text{Case}_{\text{FORS-3}}} \le \mathsf{Adv}^{\text{SM-DT-TCR-C}}_{\text{T}_2,\mathcal{A}_{3\text{-}3}}$$

$\text{Case}_{\text{FORS-4}}$ の場合，FORS の構成過程において，マークル木の根をまとめて公開鍵を生成するための tweakable ハッシュ関数 T_k(Algorithm 2 の 31 行目) に対する衝突が発生することを意味する．$\text{Case}_{\text{FORS-4}}$ が発生した場合， 同一の公開パラメータおよび tweak に対する T_k の衝突を抽出できるため， $\text{Case}_{\text{FORS-4}}$ を発生させる攻撃者から SM-DT-TCR-C を破る攻撃者 $\mathcal{A}_{3\text{-}4}$ を構成できる．一方で，T_k は SM-DT-TCR-C を満たすと仮定しているため，$\mathcal{A}_{3\text{-}4}$ の成功確率は無視可能である．したがって，$\text{Case}_{\text{FORS-4}}$ において $\mathcal{A}_3$ が勝利する確率は無視可能である．

$$\mathsf{Adv}_{\text{Case}_{\text{FORS-4}}} \le \mathsf{Adv}^{\text{SM-DT-TCR-C}}_{\text{T}_k,\mathcal{A}_{3\text{-}4}}$$

このとき，$\text{Case}_{\text{FORS}}$ で $\mathcal{A}$ が勝利する確率は以下のように

© 2026 Information Processing Society of Japan

評価できる．

$$\begin{aligned}\mathsf{Adv}_{\mathrm{Case}_{\mathrm{FORS}}} &\le \mathsf{Adv}^{\mathrm{EUF-CMA}}_{\mathrm{FORS},\mathcal{A}_3} \\ &\le \mathsf{Adv}^{\mathrm{ITSR}}_{\mathrm{MCCO},\mathcal{A}_{3\text{-}1}} + \mathsf{Adv}^{\mathrm{SM\text{-}DTOpenPRE}}_{\mathrm{T}_1,\mathcal{A}_{3\text{-}2}} \\ &\quad + \mathsf{Adv}^{\mathrm{SM\text{-}DT\text{-}TCR\text{-}C}}_{\mathrm{T}_2,\mathcal{A}_{3\text{-}3}} + \mathsf{Adv}^{\mathrm{SM\text{-}DT\text{-}TCR\text{-}C}}_{\mathrm{T}_k,\mathcal{A}_{3\text{-}4}} \end{aligned} \tag{3}$$

$\mathrm{Case}_{\mathrm{HT}}$ の場合，HT に対する偽造が成功することを意味する．HT において生成される乱数は疑似乱数生成器により与えられるが，これらは仮定から PRF を持つため，攻撃者から見て真の乱数と識別不可能である．このとき，真の乱数を用いた HT は，既存研究 [16] における FL-SL-XMSS$^{\mathrm{MT\$}}$ と等価な構成である．したがって，HT に対する偽造の成功は，FL-SL-XMSS$^{\mathrm{MT\$}}$ に対する偽造へ帰着される．よって，$\mathcal{A}$ を用いて，FL-SL-XMSS$^{\mathrm{MT\$}}$ の EUF-NAGCMA ゲームに勝利する攻撃者 $\mathcal{A}_4$ を構成できる．このとき，$\mathrm{Case}_{\mathrm{HT}}$ で $\mathcal{A}$ が勝利する確率は以下のように評価できる．

$$\mathsf{Adv}_{\mathrm{Case}_{\mathrm{HT}}} \le \mathsf{Adv}^{\mathrm{EUF\text{-}NAGCMA}}_{\mathrm{FL\text{-}SL\text{-}XMSS}^{\mathrm{MT\$}},\mathcal{A}_4} \tag{4}$$

既存研究 [16] により，FL-SL-XMSS$^{\mathrm{MT\$}}$ は EUF-NAGCMA 安全であるため，当該確率は無視可能である．したがって，HT における偽造成功確率も無視可能である．

式 (1)，式 (2)，式 (3)，式 (4) より $\mathcal{A}$ が SPHINCS$^+_{\mathrm{B}}$ に対する BEUF-CMA ゲームに勝利する確率は以下で評価できる．

$$\begin{aligned}\mathsf{Adv}^{\mathrm{BEUF\text{-}CMA}}_{\mathrm{SPHINCS}^+_{\mathrm{B}},\mathcal{A}} &\le \mathsf{Adv}^{\mathrm{PRF}}_{\mathrm{SKG},\mathcal{A}_1} + \mathsf{Adv}^{\mathrm{PRF}}_{\mathrm{MKG},\mathcal{A}_2} + \mathsf{Adv}^{\mathrm{ITSR}}_{\mathrm{MCCO},\mathcal{A}_{3\text{-}1}} \\ &\quad + \mathsf{Adv}^{\mathrm{SM\text{-}DTOpenPRE}}_{\mathrm{T}_1,\mathcal{A}_{3\text{-}2}} + \mathsf{Adv}^{\mathrm{SM\text{-}DT\text{-}TCR\text{-}C}}_{\mathrm{T}_2,\mathcal{A}_{3\text{-}3}} \\ &\quad + \mathsf{Adv}^{\mathrm{SM\text{-}DT\text{-}TCR\text{-}C}}_{\mathrm{T}_k,\mathcal{A}_{3\text{-}4}} + \mathsf{Adv}^{\mathrm{EUF\text{-}NAGCMA}}_{\mathrm{FL\text{-}SL\text{-}XMSS}^{\mathrm{MT\$}},\mathcal{A}_4}\end{aligned}$$

□

6. 実装評価

本節では，SPHINCS$^+_{\mathrm{B}}$ を実装評価する．

6.1 目的と設定

実験の目的は，任意の平文数に対して署名サイズと処理時間が個々の署名を一つずつ処理する場合（個別署名と呼称）と比べて改善されているか実証することにある．

実装は C 言語，個別署名には PQClean ライブラリの SPHINCS$^+$ を用いた．MKHF の実装では，3 節で述べられた具体的手法を SHA-2 を用いて構成する．具体的には，128-bit セキュリティレベルに対応する SPHINCS$^+$-128f および SPHINCS$^+$-128s を評価対象とした．実験環境は Windows 11 Pro 上の WSL 2（Ubuntu 24.04.2 LTS, WSL version 2.5.9.0）で，CPU は Intel Core i7-14700（14 コア / 28 スレッド，最大約 4.22 GHz），WSL2 のメモリは 7.8 GB（Swap 2 GB）である．コンパイルは GCC 13.3.0 を用い，言語仕様として `-std=c99`，最適化オプションとして `-O3` を適用した．比較対象は個別署名とする．計測は 100 回行い，その平均値を結果とする．

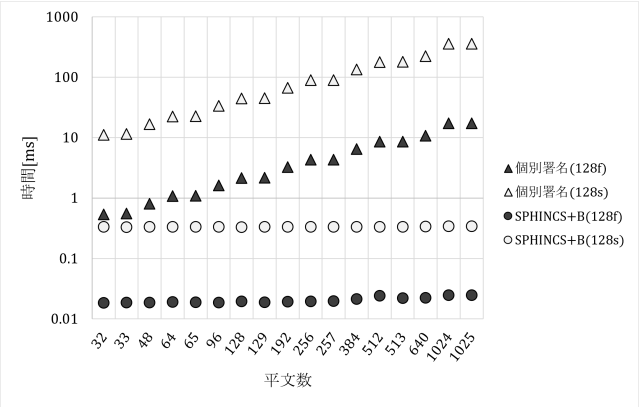


図 2: 署名生成処理時間比較

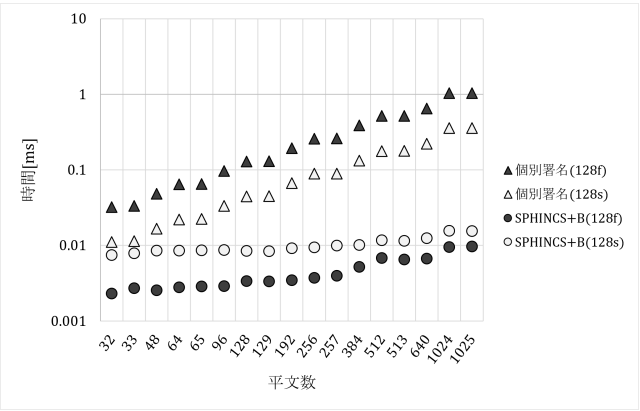


図 3: 署名検証処理時間比較

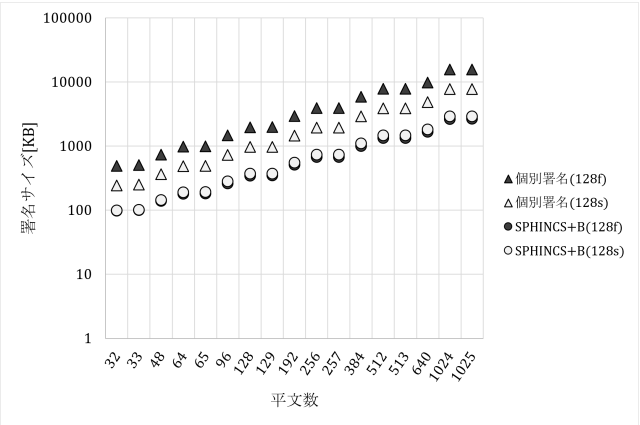


図 4: 署名サイズ比較

6.2 実験結果

SPHINCS$^+_{\mathrm{B}}$ の処理として 4 節の署名生成，署名検証の処理時間，および署名サイズを示す．まず図 2 に署名生成処理時間の結果を示す．SPHINCS$^+_{\mathrm{B}}$ は，一貫して処理時間が短縮し，この効果は平文数増加に伴い顕著となった．特に 1025 個の際は，個別署名に比べ 128f では **99.8%**，128s では **99.9%**の処理時間削減が確認された．これは，個別署名では平文ごとに実行される FORS 木構築と HT 署名生成を 1 回に集約できるためである．次に，図 3 に署名検証の処理時間の結果を示す．SPHINCS$^+_{\mathrm{B}}$ は，一貫して処理時間が

© 2026 Information Processing Society of Japan

短縮し，この効果は平文数増加に伴い顕著となった．特に 1025 個の際は，個別署名に比べ 128f では **99.0%**，128s では **95.7%**の処理時間を削減が確認された．これは，個別署名では HT の署名検証が平文数回分実行されていたのに対して，SPHINCS^+_B は FORS 木の根の値が一致するか確認したのち HT の署名検証が 1 回のみ実行されているためである．図 4 に署名サイズの結果を示す．SPHINCS^+_B はすべての条件で個別署名より署名サイズが小さく，平文数 1025 個の場合，128f では **83.0%**，128s では **62.3%** の削減を達成した．これは，平文ごとに必要だった HT 署名を平文集合全体で 1 個に集約できるためである．なお，上述した結果を踏まえたブロックチェーンへの応用を付録 A.3 に示す．

7. まとめ

本稿では，ハッシュベース署名の構造を活用して署名をバッチ化する SPHINCS^+_B を提案した．SPHINCS^+_B の設計にあたり，新たに平文集合に対する鍵付きハッシュ関数 MKHF を導入し，FORS におけるマークル木の葉の選択を抽象化することで，同一の公開鍵に対応する複数の FORS 署名を一括して構成可能になる．また，SPHINCS^+_B の安全性を MKHF およびハッシュ関数の衝突困難性に基づく新たな技法で証明した．さらに，C 言語による実装評価の結果，SPHINCS$^+$ を個別に実行する場合と比較して，検証時間を最大 95.7%，署名サイズを最大 62.8%削減できることを示した．今後の課題は，Ethereum などブロックチェーン環境における SPHINCS^+_B の実装評価と，MKHF の効率的な構成の検討である．

参考文献

[1] Aumasson, J.-P., Bernstein, D. J., Beullens, W., Dobraunig, C., Eichlseder, M., Fluhrer, S., Gazdag, S.-L., Hülsing, A., Kampanakis, P., Kölbl, S., Lange, T., Lauridsen, M. M., Mendel, F., Niederhagen, R., Rechberger, C., Rijneveld, J., Schwabe, P. and Westerbaan, B.: SPHINCS+ – Submission to the 3rd Round of the NIST Post-Quantum Cryptography Project (2022). Version 3.1.

[2] Ott, D., Peikert, C. and et al.: Identifying Research Challenges in Post Quantum Cryptography Migration and Cryptographic Agility, *CoRR*, Vol. abs/1909.07353 (2019).

[3] Marchesi, L., Marchesi, M., Destefanis, G., Barabino, G. and Tigano, D.: Design Patterns for Gas Optimization in Ethereum, *Proc. of IWBOSE 2020*, pp. 9–15 (2020).

[4] Aguilar-Melchor, C., Albrecht, M. R., Bailleux, T., Bindel, N., Howe, J., Hülsing, A., Joseph, D. and Manzano, M.: Batch Signatures, Revisited, *Proc. of CT-RSA 2024*, LNCS, Vol. 14643, Springer, pp. 163–186 (2024).

[5] Benjamin, D.: Batch Signing for TLS, Internet-Draft draft-ietf-tls-batch-signing-00, Internet Engineering Task Force (2020). Work in Progress.

[6] Aumasson, J.-P. and Endignoux, G.: Improving stateless hash-based signatures, *Proc. of CT-RSA 2018*, Springer, pp. 219–242 (2018).

[7] Zhang, K., Cui, H. and Yu, Y.: SPHINCS-α: A Compact Stateless Hash-Based Signature Scheme, *IACR Cryptol. ePrint Arch.*, Vol. 2022, pp. 1–29 (2022).

[8] Hülsing, A., Kudinov, M., Ronen, E. and Yogev, E.: SPHINCS+ C: compressing SPHINCS+ with (almost) no cost, *2023 IEEE Symposium on Security and Privacy (SP)*, IEEE, pp. 1435–1453 (2023).

[9] Consigny, N.: SPHINCS minus : Efficient Stateless Post-Quantum Signature Verification on the EVM (2026). https://ethresear.ch/t/sphincs-minus-efficient-stateless-post-quantum-signature-verification-on-the-evm/25165.

[10] Zhu, F., Yi, X., Abuadbba, A., Luo, J., Nepal, S. and Huang, X.: Efficient hash-based redactable signature for smart grid applications, *Proc. of ESORICS 2022*, Springer, pp. 554–573 (2022).

[11] Fregly, A., Harvey, J., Kaliski Jr., B. S. and Sheth, S.: Merkle Tree Ladder Mode: Reducing the Size Impact of NIST PQC Signature Algorithms in Practice, *Proc. of CT-RSA 2023*, LNCS, Vol. 13871, Springer, pp. 415–441 (2023).

[12] 三好茜音，高橋　康，矢内直人：証明書の柔軟な更新に向けた完全バッチ署名の提案 (2026). SCIS2026.

[13] Kysil, R., Seres, I. A., Kutas, P. and Kelecsényi, N.: poqeth: Efficient, post-quantum signature verification on Ethereum, *Proc. of AsiaCCS 2025*, ACM, pp. 327–343 (2025).

[14] SouadN' Ait Bella, Abdulsalam, Y. S., Hedabou, M.: Efficient Implementation of Post-quantum Signature Algorithms for Next Generation of Blockchains, *ICISC 2025*, Springer, pp. 21–42 (2025).

[15] Drake, J., Khovratovich, D., Kudinov, M. and Wagner, B.: Hash-based multi-signatures for post-quantum ethereum, *IACR Communications in Cryptology*, Vol. 2, No. 1 (2025).

[16] Barbosa, M., Dupressoir, F., Hülsing, A., Meijers, M. and Strub, P.-Y.: A tight security proof for SPHINCS+, formally verified, *ASIACRYPT 2024*, Springer, pp. 35–67 (2024).

© 2026 Information Processing Society of Japan

付　　録

A.1 追加の準備

A.1.1 鍵付きハッシュ関数

本稿では，鍵付きハッシュ関数に対して，ITSR を扱う．

定義 3 (ITSR) 鍵付きハッシュ関数が ITSR を持つとは，以下に定義するゲームにおいて，攻撃者 $\mathcal{A}$ の勝利する確率 $\mathsf{Adv}^{\mathrm{ITSR}}_{\mathrm{KHF},\mathcal{A}}$ が無視可能であることをいう．

Setup : 挑戦者 C は $Q = \emptyset$ を初期化する; **Oracle]** :$\mathcal{A}$ は任意の平文 μ を選び C に要求する．C は鍵 k をランダムに生成し, (k, μ) を Q に追加した上で, 鍵 k を $\mathcal{A}$ に返す; **Collision-Test** :$\mathcal{A}$ は (k^*, μ^*) を出力する．このとき, $(k^*, \mu^*) \notin Q$ かつ $\mathrm{MAP}(\mathrm{KHF}(k^*, \mu^*)) \in \{\mathrm{MAP}(\mathrm{KHF}(k_i, \mu_i)) \mid (k_i, \mu_i) \in Q\}$ が成り立つとき $\mathcal{A}$ が勝利する．ここで MAP は鍵付きハッシュ関数の出力をターゲット集合へ写像する関数であり，ITSR の定義において攻撃対象となる要素集合を規定する．

A.1.2 tweakable ハッシュ関数

SPHINCS$^+$ では入力長に応じて異なる THF を用いるため，THF の集合（THF Collection; THFC）を考える．入力平文の取りうる長さの集合を Λ とすると，THFC は $\mathrm{THFC} = \{\mathrm{THF}_\lambda : \mathcal{P} \times \mathcal{T} \times \mathcal{M} \to \mathcal{Y}\}_{\lambda \in \Lambda}$ と定義される．本稿では，tweakable ハッシュ関数に対して，Single-function・Multi-target・Distinct-tweak（SM-DT）設定における，Target-Collision Resistance（SM-DT-TCR）および同一公開パラメータを共有する tweakable ハッシュ関数の利用を考慮した拡張版である SM-DT-TCR-C を扱う．また，Decisional Second-Preimage Resistance（SM-DT-DSPR）および Opening-Preimage Resistance（SM-DT-OpenPRE）についても扱う．

定義 4 (SM-TCR) tweakable ハッシュ関数が SM-DT-TCR を持つとは, 以下に定義するゲームにおいて, 最大 q 個のターゲットを持つ任意の攻撃者 $\mathcal{A}$ の勝利確率 $\mathsf{Adv}^{\text{SM-DT-TCR}}_{\mathrm{THF},\mathcal{A}}$ が無視可能であることをいう．**Setup**:λ を与えられた挑戦者 C は公開パラメータ p をランダムに生成し，$Q = \emptyset$ を初期化する; **Query**:$\mathcal{A}$ は任意の tweak tw と平文 x を選び，C に要求する．C は $y = \mathrm{THF}(p, tw, x)$ を計算し，(tw, x) を Q 追加して y を返す; **CollisionTest]**:$\mathcal{A}$ は (i^*, x^*) を出力する．$i < |Q_1|$ のとき $Q_1[i] = (tw_i, x_i)$ とする．このとき，以下が全て成り立つ場合，$\mathcal{A}$ は勝利する:$i^* < q$，$(tw_{i^*}, x_{i^*}) \in Q_1$，$x_{i^*} \neq x^*$，$\mathrm{THF}(p, tw_{i^*}, x_{i^*}) = \mathrm{THF}(p, tw_{i^*}, x^*)$.

定義 5 (SM-DT-TCR-C) tweakable ハッシュ関数が SM-DT-TCR-C を持つとは，以下に定義するゲームにおいて，最大 q 個のターゲットを持つ任意の攻撃者 $\mathcal{A}$ の勝利確率 $\mathsf{Adv}^{\text{SM-DT-TCR-C}}_{\mathrm{THF},\mathcal{A}}$ が無視可能であることをいう．

Setup: 挑戦者 C は公開パラメータ p をランダムに生成し，$Q_1 = \emptyset$，$Q_2 = \emptyset$ を初期化する; **Oracle**:$\mathcal{A}$ は任意の tweak tw と平文 x を選び, C に要求する．C は $y = \mathrm{THF}(p, tw, x)$ を計算し，(tw, x) を Q_1 追加して y を返す; **CollectionOracle**:$\mathcal{A}$ は任意の tweak tw と平文 x を選び，補助オラクルに要求する．補助オラクルは $y = \mathrm{THFC}_{|x|}(p, tw, x)$ を計算し，tw を Q_2 追加して y を返す; **CollisionTest**:$\mathcal{A}$ は (i^*, x^*) を出力する．$i < |Q_1|$ のとき $Q_1[i] = (tw_i, x_i)$ とする．このとき, 以下が全て成り立つ場合, $\mathcal{A}$ は勝利する:$i^* < q$, $(tw_{i^*}, x_{i^*}) \in Q_1$, $tw_{i^*} \notin Q_2$，$x_{i^*} \neq x^*$，$\mathrm{THF}(p, tw_{i^*}, x_{i^*}) = \mathrm{THF}(p, tw_{i^*}, x^*)$.

定義 6 (SM-DT-DSPR) tweakable ハッシュ関数が SM-DT-DSPR を持つとは，以下に定義するゲームにおいて，最大 q 個のターゲットを持つ任意の攻撃者 $\mathcal{A}$ の勝利確率 $\mathsf{Adv}^{\text{SM-DT-DSPR}}_{\mathrm{THF},\mathcal{A}} := |\Pr[\mathcal{A} \text{ wins SM-DT-DSPR}] - 1/2|$ が無視可能であることをいう．**Setup**: 挑戦者 C は公開パラメータ p をランダムに生成し，$Q = \emptyset$ を初期化する; **Oracle**:$\mathcal{A}$ は任意の tweak tw と入力 x を選び，C に要求する．C は $y = \mathrm{THF}(p, tw, x)$ を計算し，(tw, x) を Q に追加して y を返す; **CollisionTest**:$\mathcal{A}$ は (i^*, b^*) を出力する．$i < |Q|$ のとき $Q[i] = (tw_i, x_i)$ とする．ここで b は，$x' \neq x_{i^*}$ かつ $\mathrm{THF}(p, tw_{i^*}, x') = \mathrm{THF}(p, tw_{i^*}, x_{i^*})$ が成り立つ x' が存在する場合に $b = 1$，存在しない場合に $b = 0$ とする．このとき，以下が全て成り立つとき，$\mathcal{A}$ は勝利する:$i^* < q$, $(tw_{i^*}, x_{i^*}) \in Q$，$b = b^*$.

定義 7 (SM-DT-OpenPRE) tweakable ハッシュ関数が SM-DT-OpenPRE を持つとは，以下に定義するゲームにおいて，最大 q 個のターゲットを持つ任意の攻撃者 $\mathcal{A}$ の勝利確率 $\mathsf{Adv}^{\text{SM-DT-OpenPRE}}_{\mathrm{THF},\mathcal{A}}$ が無視可能であることをいう．**Setup**: 挑戦者 C は公開パラメータ p をランダムに生成し，$Q_1 = \emptyset$，$Q_2 = \emptyset$ を初期化する．$\mathcal{A}$ は tweak の集合 **TW** を出力する．各 $tw_i \in \mathbf{TW}$ に対して，ランダムに x_i を選び，$y_i = \mathrm{THF}(p, tw_i, x_i)$ を計算する．x_i を Q_1 に追加し，出力の集合 $\mathbf{Y} = \{y_i | i\}$ を攻撃者 $\mathcal{A}$ に返す; **Oracle**:$\mathcal{A}$ はインデックス i を選び，C に要求する．C は i を Q_2 に追加し対応する入力 x_i を返す; **CollisionTest**:$\mathcal{A}$ は (i^*, x^*) を出力する．$i < |Q_1|$ のとき $Q_1[i] = x_i$ とする．このとき，以下が全て成り立つ場合，$\mathcal{A}$ は勝利する:$i^* < t$，$x_{i^*} \in Q_1$，$i^* \notin Q_2$，$\mathrm{THF}(p, tw_{i^*}, x^*) = y_{i^*}$.

A.1.3 ディジタル署名

ディジタル署名は KeyGen，Sign，Verify の 3 つのアルゴリズムで構成される．**KeyGen** はセキュリティパラメータ λ を入力として，公開鍵 PK と秘密鍵 SK とを出力する; **Sign** は秘密鍵 SK と平文 μ を入力として，署名 σ を出力する; **Verify** は公開鍵 PK，平文 μ，署名 σ を入力として，ビット $\{0, 1\}$ を出力する．

ディジタル署名の正当性は，任意の $(\mathrm{PK}, \mathrm{SK}) \leftarrow \mathrm{KeyGen}(1^\lambda)$ と平文 μ に対し, $\mathrm{Verify}(\mathrm{PK}, \mu, \mathrm{Sign}(\mathrm{SK}, \mu)) = 1$ とする．

定義 8 (EUF-CMA) ディジタル署名 S が EUF-CMA（Exis-

© 2026 Information Processing Society of Japan

tential Unforgeability under Chosen Message Attack）を持つとは, 以下に定義するゲームにおいて, 任意の攻撃者 $\mathcal{A}$ の勝利確率 $\mathsf{Adv}^{\text{EUF-CMA}}_{\text{S},\mathcal{A}}(n)$ が λ に対して無視可能であることを言う.

Setup: セキュリティパラメータ λ を与えられた 挑戦者 C は KeyGen(n) を実行し，出力された鍵ペアのうち公開鍵 PK を $\mathcal{A}$ に渡す．また，$\mathcal{M} = \emptyset$ とする; **Oracle**:$\mathcal{A}$ は任意の平文 μ_i について署名を要求する．C は Sign(SK, μ_i) を実行し，出力である署名 σ_i を返す．また，$\mathcal{M} = \mathcal{M} \cup \{\mu_i\}$ とする; **CollisionTest**:$\mathcal{A}$ は (μ^*, σ^*) を出力する．このとき，$\mu^* \notin \mathcal{M}$，Verify(PK, μ^*, σ^*) = 1 が全て成り立つ場合，$\mathcal{A}$ は勝利する.

定義 9 (EUF-NAGCMA) ディジタル署名 S が EUF-NAGCMA（a non-adaptive, generic version of the EUF-CMA）を持つとは, 以下に定義するゲームにおいて, 任意の攻撃者 $\mathcal{A}$ の勝利確率 $\mathsf{Adv}^{\text{EUF-NAGCMA}}_{\text{S},\mathcal{A}}(n)$ が n に対して無視可能であることを言う.

Setup: セキュリティパラメータ λ を与えられた 挑戦者 C は KeyGen(n) を実行し，出力された鍵ペアのうち公開鍵 PK を $\mathcal{A}$ に渡す; **Oracle**: $\mathcal{A}$ は署名を要求する任意の平文集合 $\mathcal{M} = \{\mu_0, \ldots, \mu_{N-1}\}$ を出力する．C は各 $\mu_i \in \mathcal{M}$ に対して $\sigma_i \leftarrow$ Sign(SK, μ_i) を計算し，署名集合 $\mathcal{S} = \{\sigma_0, \ldots, \sigma_{N-1}\}$ を $\mathcal{A}$ へ返す; **CollisionTest**:$\mathcal{A}$ は (μ^*, σ^*, i^*) を出力する．このとき，$\mu^* \neq m_{i^*}$，Verify(PK, μ^*, σ^*) = 1，$0 \leq i^* < N$ が全て成り立つ場合，$\mathcal{A}$ は勝利する.

A.2 SPHINCS$^+_{\text{B}}$ の正当性と効率性の証明

A.2.1 正当性

定理 2 (正当性) SPHINCS$^+_{\text{B}}$ は正当性を持つ.

Proof. Algorithm 2，Algorithm 3 を比較すると，以下が成り立つ．Algorithm 2 の 3-4 行目と Algorithm 3 の 2-3 行目では乱数 S_R，公開鍵に含まれるシード値，平文集合 M から FORS が処理できる固定長平文 MD と，どの FORS インスタンスを使うかを示す index を算出し，FORS のインスタンスを示す index を，idx_tree，idx_leaf に分解する．両アルゴリズムは同一の入力 S_R, PK.seed, PK.root, M に対して MCCO を行うため，得られる MD，idx_tree，idx_leaf 値は Bsign と Bverify において常に一致する．また，FORS に関しては，Algorithm 2 の 6 行目の FORS.BSign により生成された署名から，Algorithm 3 の 5-9 行目 FORS.pkFromSig を用いて同一の PK_{FORS} が復元される．FORS.BSign は，FORS.Sign と同一の処理を行うものを，複数の平文に対して並列に適用したものであるため，FORS の正当性からこの性質が成立する．最後に，HT に関しても，Algorithm 2 の 8 行目の HT.Sign により生成された署名は，Algorithm 3 の 11 行目の HT.Verify によって必ず受理されることが HT の正当性から成立する．よって，SPHINCS$^+_{\text{B}}$ は正当性を満たす. □

A.2.2 効率性

定理 3 (効率性) SPHINCS$^+_{\text{B}}$ は効率性を持つ.

Proof. $N > \sqrt{(|S_R| + |S_{\text{FORS}}| + |S_{\text{HT}}|)/(|S_R| + |S_{\text{HT}}|)}$ において，平文集合 $M = \{\mu_i\}_{i=0}^{N-1}$ を考える．このとき，BSign(SK,M) により得られる署名を $S = (S_R, S_{\text{FORS}}, S_{\text{HT}})$，また，Sign(SK, μ_i) により得られる署名を $S'_i = (S'_{R,i}, S'_{\text{FORS},i}, S'_{\text{HT},i})$ とする．ここで，署名サイズは

$$|S| = |S_R| + |S_{\text{FORS}}| + |S_{\text{HT}}|$$
$$|S'| = \sum_{i=0}^{N-1} (|S'_{R,i}| + |S'_{\text{FORS},i}| + |S'_{\text{HT},i}|)$$

となる．S_R および S_{HT} は単一平文に対する署名と同一のサイズのため $|S_R| = |S'_{R,i}|, |S_{\text{HT}}| = |S'_{\text{HT},i}|$ が成り立つ．また，S_{FORS} は各平文 μ_i 対応する FORS 署名 N 個と，平文集合 M の FORS 署名 1 個から構成されるため，合計 $N+1$ 個の FORS 署名を含む．一方，$S'_{\text{FORS},i}$ は平文 μ'_i に対応する FORS 署名 1 個から構成される．したがって，$|S_{\text{FORS}}| = (N+1)|S'_{\text{FORS},i}|$ である．よって，

$$\begin{aligned}|S'| &= \sum_{i=0}^{N-1} (|S'_{R,i}| + |S'_{\text{FORS},i}| + |S'_{\text{HT},i}|) \\ &= N(|S_R| + |S_{\text{HT}}|) + \frac{N}{N+1}|S_{\text{FORS}}| \\ &= |S| + (N-1)(|S_R| + |S_{\text{HT}}|) - \frac{1}{N+1}|S_{\text{FORS}}|\end{aligned}$$

ここで，$|S'| - |S| = (N-1)(|S_R| + |S_{\text{HT}}|) - \frac{1}{N+1}|S_{\text{FORS}}|$ である．したがって，$|S'| > |S|$ が成り立つ条件は $(N-1)(|S_R| + |S_{\text{HT}}|) > \frac{1}{N+1}|S_{\text{FORS}}|$ である．この不等式を変形すると，

$$N > \sqrt{\frac{|S_R| + |S_{\text{FORS}}| + |S_{\text{HT}}|}{|S_R| + |S_{\text{HT}}|}}$$

を得る．したがって，上記条件を満たす任意の N に対して $|S| < |S'|$ が成り立つため， SPHINCS$^+_{\text{B}}$ は署名サイズの観点で効率性を満たす．一方，各平文 μ_i に対して個別に Sign(SK, μ_i) を実行する場合，FORS 構成および HT 構成をそれぞれ N 回実行する必要がある．したがって，SPHINCS$^+_{\text{B}}$ では HT 構成の実行回数を N 回から 1 回へ削減できるとともに，FORS 処理についても複数の平文に対して共有可能な計算を利用できる．そのため，個別に署名を生成する場合と比較して計算量が削減されることから，計算量の観点でも効率性を有する.

□

A.3 SPHINCS$^+_{\text{B}}$ のブロックチェーンへの応用

SPHINCS$^+_{\text{B}}$ はブロックチェーンにおける処理を示すトランザクションについて，その正しさを検証するコンセンサスプロトコルへの導入が期待される．コンセンサスプロトコルでは，実際に処理を実行したユーザは自らのトランザクションに署名した後，バリデータと呼ばれる主体がその

© 2026 Information Processing Society of Japan

検証を実行する．このとき，最近のプロトコル[*4]ではバリデータも自ら署名を生成するようになっている．このため，バリデータの数が増えるにつれ，これらの署名の個別の生成と検証も求められる．耐量子計算機暗号ではこれら一連の負荷がボトルネックとなることが予想される．

これに対し，$\text{SPHINCS}^{+}_{\text{B}}$ を導入することで，複数のトランザクションを平文集合と見做してバッチ署名を生成する．具体的には，バリデータは各トランザクションに対してまず FORS 署名を生成し，その FORS 公開鍵に対して HT 署名を生成する．これにより，複数のトランザクションを一つのバッチ署名として扱える．6 節で示した通り $\text{SPHINCS}^{+}_{\text{B}}$ は SPHINCS^{+} の個別の利用よりも大幅な性能改善が期待できることから，$\text{SPHINCS}^{+}_{\text{B}}$ により現行のブロックチェーンの仕様を，その効率を損なうことなく耐量子計算機暗号に移行することが期待できる．より詳細な検討は今後の課題である．

[*4] https://ethereum.org/developers/docs/consensus-mechanisms/pos/attestations/?utm_source=chatgpt.com

© 2026 Information Processing Society of Japan